CURSO TÉCNICO OFICIAL · 20 HORAS

DOCKER FUNDAMENTALS



Módulo 1: Introducción a Contenedores, Arquitectura del Motor, Seguridad e Imágenes
Jornada 1 de 4 · 5 Horas Lectivas (20h totales)

El Reto del Despliegue: Del Monolito a la Matriz Matricial

La Matriz del Infierno (N x M)

- **Diversidad de Runtimes:** Coexistencia forzada de Node.js, Python, Java y Go con versiones en conflicto.
- **Librerías Compartidas:** Dependencias del sistema (glibc, OpenSSL) que rompen servicios al actualizar el host.
- **Entornos Heterogéneos:** Discrepancias críticas entre el portátil local, el servidor de QA y producción.
- **Deriva de Configuración:** Modificaciones manuales no documentadas en servidores ("servidores copo de nieve").

La Solución de Contenedores

- **Inmutabilidad:** La aplicación se empaqueta junto a todo su entorno de ejecución exacto y probado.
- **Portabilidad Absoluta:** Garantía OCI: si arranca en local, arrancará de forma idéntica en cualquier nube.
- **Arranque en Milisegundos:** Sin arranque de sistema operativo virtual; inicia como un proceso nativo del host.
- **Alta Densidad:** Cientos de microservicios en el mismo hardware sin sobrecarga de hipervisores redundantes.

PRINCIPIO CLAVE: Un contenedor no virtualiza hardware ni emula instrucciones de CPU. Es un proceso aislado ejecutado directamente sobre el kernel de Linux compartiendo sus recursos con coste cero de emulación.

ARQUITECTURA COMPARADA

Máquinas Virtuales vs Contenedores: Arquitectura Interna

Máquinas Virtuales (Hardware)

- **Nivel de Abstracción:** Emulación de hardware físico mediante hipervisor (KVM, ESXi, Hyper-V).
- **Sobrecarga de SO:** Cada VM requiere su propio kernel, módulos y demonios del sistema completo.
- **Uso de Memoria:** Consumo base de gigabytes por instancia antes de arrancar la aplicación.
- **Tiempo de Arranque:** Minutos (proceso completo de POST de BIOS virtual, inicio de kernel y systemd).
- **Aislamiento:** Fuerte aislamiento asistido por hardware (VT-x, AMD-V, SLAT).

Contenedores (Sistema Operativo)

- **Nivel de Abstracción:** Aislamiento a nivel de procesos mediante primitivas nativas del kernel Linux.
- **Kernel Compartido:** Todos los contenedores utilizan el kernel del host sin kernels huéspedes.
- **Uso de Memoria:** Megabytes despreciables de sobrecarga; solo consume la memoria de la aplicación.
- **Tiempo de Arranque:** Milisegundos (equivalente a invocar un fork/clone de un proceso nativo).
- **Aislamiento:** Aislamiento lógico vía Namespaces, Control Groups y perfiles Seccomp/AppArmor.

BAJO EL CAPÓ

¿Qué es un Contenedor? El Secreto del Kernel de Linux

La Verdad Técnica sobre Contenedores

- **No hay entidad contenedor:** En el código fuente del kernel no existe ninguna estructura llamada "container".
- **Proceso Ordinario:** Un contenedor es un proceso estándar de Linux (`task_struct`) con visibilidad y recursos limitados.
- **Los Tres Pilares:** Namespaces (lo que ve), Cgroups (lo que consume) y Capabilities/LSM (lo que puede ejecutar).
- **Rendimiento Nativo:** La CPU ejecuta las instrucciones del contenedor en Ring 3 con acceso directo al bus de memoria.

COMPROBACIÓN DE PROCESOS EN HOST Y CONTENEDOR

```
# 1. Arrancar un contenedor web en segundo plano
$ docker run -d --name web-demo nginx:alpine
7f8a9b2c3d4e...

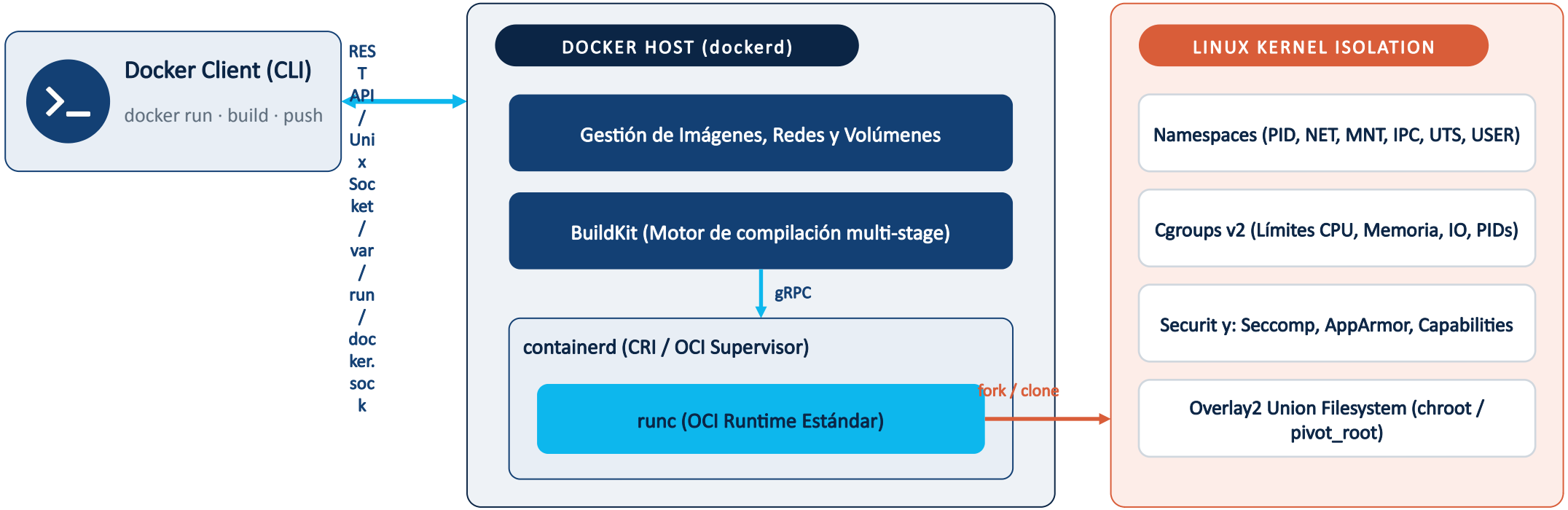
# 2. Obtener el PID asignado por el host
$ docker inspect -f '{{.State.Pid}}' web-demo
42150

# 3. Comprobar que en el host es un proceso normal
$ ps -fp 42150
UID    PID   PPID  CMD
root  42150 42100  nginx: master process /usr/sbin/nginx

# 4. Comprobar que dentro del contenedor se ve como PID 1
$ docker exec web-demo ps -ef
UID   PID  PPID  CMD
root    1     0  nginx: master process /usr/sbin/nginx
```

ARQUITECTURA INTERNA

Arquitectura del Motor Docker: Desacoplamiento OCI



Docker-Engine vs Docker.io: Paquetes y Distribuciones

Docker-CE / Docker-Engine (Oficial)

- **Origen Oficial:** Mantenido directamente por Docker Inc en repositorios dedicados para Debian/Ubuntu/RHEL.
- **Versiones Al Día:** Incluye los últimos parches de seguridad del kernel, mejoras de OCI y rendimiento.
- **Plugins Modernos:** Paquetes `docker-buildx-plugin` y `docker-compose-plugin` (v2 integrado en el CLI).
- **Estándar Profesional:** La opción recomendada para entornos de desarrollo corporativo y producción.

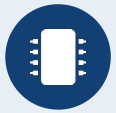
Docker.io (Distribuciones Linux)

- **Origen Distribución:** Mantenido por los empaquetadores de Debian/Ubuntu para compatibilidad estática.
- **Versiones Congeladas:** Suele incluir versiones antiguas con vulnerabilidades corregidas tardíamente.
- **Sin Plugins Nativos:** Carece habitualmente de Buildx avanzado y requiere el viejo script de Compose v1.
- **Riesgos de Compatibilidad:** Fallo al interpretar directivas de build modernas y drivers de red recientes.

AVISO DE INSTALACIÓN: Evitar `apt-get install docker.io`. En entornos corporativos configurar siempre el repositorio oficial `https://download.docker.com/linux/ubuntu` con clave GPG e instalar el paquete `docker-ce`.

SEGURIDAD

Filosofía de Seguridad: Defensa en Profundidad



NAMESPACES (Visibilidad / Aislamiento)

Determinan lo que un proceso puede VER en el sistema

PID	Aísla el árbol de procesos (el contenedor tiene su propio PID 1).
NET	Aísla interfaces de red, tablas de enrutamiento y puertos TCP/UDP.
MNT	Aísla puntos de montaje de sistemas de archivos (raíz aislada).
IPC	Aísla colas de mensajes POSIX y segmentos de memoria compartida.
UTS	Aísla nombre de host (hostname) y dominio NIS independiente.
USER	Mapea UID/GID del contenedor a usuarios sin privilegios del host.



CONTROL GROUPS / CGROUPS (Límites)

Determinan lo que un proceso puede USAR de hardware

CPU Quota	Limita cuotas de ciclos CPU por periodo (--cpus="1.5").
RAM Limit	Límite estricto de memoria (--memory="512m"); dispara OOM Killer.
BlkIO / Disk	Control de ancho de banda y operaciones de E/S por segundo en disco.
PIDs Limit	Límite de subprocesos (--pids-limit=100) para evitar fork-bombs.
Network Pri	Priorización de tráfico de red y etiquetado con net_cls / net_prio.

AISLAMIENTO DEL KERNEL

Linux Namespaces: Creando la Ilusión de Máquina Privada

PID Namespace (Procesos)

Aísla el árbol de procesos. El proceso del contenedor se convierte en PID 1 dentro de su espacio, sin ver ni poder enviar señales a los procesos del host.

NET Namespace (Red)

Aísla interfaces de red (lo, eth0), tablas de enrutamiento y puertos TCP/UDP. Dos contenedores pueden escuchar en el puerto 80 sin colisionar.

MNT Namespace (Montajes)

Aísla la jerarquía de directorios. Cada contenedor tiene su propia raíz `/` provista por el UnionFS sin acceso al sistema de archivos del anfitrión.

IPC Namespace (Memoria)

Aísla colas de mensajes POSIX, semáforos SysV y segmentos de memoria compartida, impidiendo que procesos de diferentes contenedores se comuniquen indebidamente.

UTS Namespace (Host & Dominio)

Permite definir un hostname y nombre de dominio NIS independiente para cada contenedor sin alterar la configuración del servidor host.

USER Namespace (Usuarios)

Mapea el UID 0 (root) del contenedor a un usuario sin privilegios en el host (ej. UID 100000), base fundamental de la arquitectura Rootless.

Control Groups (Cgroups): Límites y Protección del Host

Protección Frente al "Vecino Ruidoso"

- **Contención DoS:** Evita que un proceso desbocado colapse el hardware compartido del servidor.
- **Límite de RAM (`--memory`):** Tope estricto de memoria física. Si se excede, el OOM Killer actúa localmente.
- **Cuotas de CPU (`--cpus`):** Asignación de periodos de CPU mediante el CFS (Completely Fair Scheduler).
- **Límite de PIDs (`--pids-limit`):** Inmunidad total frente a ataques de bomba de bifurcación (fork-bombs).

APLICACIÓN DE LÍMITES EN DOCKER CLI

```
# Arrancar contenedor con límites estrictos de hardware
$ docker run -d --name app-limitada \
    --memory="512m" \
    --cpus="1.5" \
    --pids-limit=100 \
    nginx:alpine

# Monitorizar consumo de recursos en tiempo real
$ docker stats --no-stream app-limitada
NAME           CPU %   MEM USAGE / LIMIT   MEM %
app-limitada   0.02%   4.1MiB / 512MiB     0.80%

# Inspeccionar el valor exacto en el kernel (Cgroups v2)
$ cat /sys/fs/cgroup/system.slice/docker-.../memory.max
536870912
```

SEGURIDAD AVANZADA

Capabilities de Linux y Modo Rootless: Mínimo Privilegio

Principio de Mínimo Privilegio

- **Fraccionar a Root:** Sustituir el poder absoluto de UID 0 por capacidades granulares del kernel.
- **Eliminar Riesgos:** Retirar capabilities para impedir montaje de discos, acceso a memoria física o manipulación de red.
- **Antipatrón `--privileged`:** ¡Prohibido en producción! Desactiva todos los filtros de seguridad del host.
- **Modo Rootless:** Ejecutar el daemon dockerd íntegramente bajo un usuario sin privilegios en el host.

GESTIÓN PROFESIONAL DE CAPABILITIES

```
# Buena práctica: Retirar todo y conceder lo mínimo
$ docker run -d --name web-secure \
    --cap-drop=ALL \
    --cap-add=NET_BIND_SERVICE \
    nginx:alpine

# Comprobar que no puede alterar interfaces de red
$ docker exec web-secure ip link add dummy0 type dummy
RTNETLINK answers: Operation not permitted
```

ALERTA DE SEGURIDAD: El uso de `docker run --privileged` otorga todas las capabilities, deshabilita AppArmor y mapea todos los dispositivos de `/dev`. Constituye una brecha de escape del contenedor directa.

¿Qué es una Imagen en Docker? La Plantilla Inmutable

Propiedades de una Imagen OCI

- **Plantilla de Solo Lectura:** Contiene el código, runtime, dependencias del sistema y configuración.
- **Inmutabilidad Estricta:** Una imagen nunca se modifica; los cambios generan una nueva imagen versionada.
- **Identificador Criptográfico:** Cada capa e imagen se referencia mediante un hash SHA-256 incorruptible.
- **Estándar Abierto OCI:** Compatible entre Docker, Podman, Containerd, CRI-O y plataformas cloud.

Anatomía de Referencia de Imagen

- **Formato Completo:** `[registry]/[namespace]/[repositorio]:[tag]`
- **Ejemplo Real:** `docker.io/library/nginx:1.25.3-alpine`
- **Valores por Defecto:** Registry por defecto: Docker Hub. Tag por defecto: `:latest`.
- **Peligro de `:latest`:** ¡Antipatrón en producción! Puede introducir cambios incompatibles sin previo aviso.

REGLA DE PRODUCCIÓN: Nunca desplegar imágenes con el tag `:latest`. Utilizar siempre etiquetas inmutables con control de versiones semántico (ej. `node:20.11.1-alpine3.19`) o referenciar directamente por Digest SHA-256.

Características de una Imagen: Capas y Caché Overlay2

PILA DE CAPAS OVERLAY2 (Estructura en Disco)

UPPERDIR (R/W)	Capa de contenedor: ficheros creados, modificados o borrados (whiteout). Se destruye con el contenedor.
CAPA 3 (R/O)	COPY . /app & ENTRYPOINT (Ficheros de aplicación y binarios)
CAPA 2 (R/O)	RUN npm install / pip install (Dependencias y librerías del runtime)
CAPA 1 (R/O)	FROM node:20-alpine (Base OS minimalista, libmusl, shell / bin/sh)

OverlayFS Union Mount →

MERGED VIEW (Vista Unificada)

Punto de montaje `/` que ve el proceso dentro del contenedor. Las capas de sólo lectura se superponen; si un fichero existe en el Upperdir, enmascara al del Lowerdir.

MECÁNICA COPY-ON-WRITE (CoW)

- LECTURA: Acceso directo ultra rápido a la capa inferior.
- ESCRITURA: Al modificar un archivo existente, el kernel lo duplica primero a Upperdir y luego aplica el cambio.
- BORRADO: Se genera un marcador whiteout en Upperdir.

¿Qué es un Contenedor? La Instancia Viva y Copy-on-Write

La Relación Clase - Objeto

- **Clase vs Instancia:** La imagen es la definición estática inmutable; el contenedor es la instancia viva.
- **Capa Superior R/W:** Fina capa de lectura/escritura asociada exclusivamente al contenedor específico.
- **Copy-on-Write (CoW):** Los archivos originales jamás se tocan; se duplican a la capa superior solo al modificarse.
- **Naturaleza Efímera:** Al eliminar el contenedor (`docker rm`), su capa R/W se destruye; la imagen no cambia.

DEMOSTRACIÓN DE COPY-ON-WRITE CON DOCKER DIFF

```
# 1. Arrancar un contenedor interactivo
$ docker run -it --name demo-cow ubuntu:22.04 bash

# 2. Modificar un archivo del sistema dentro del contenedor
root@a1b2c3d4:/# echo "Host modificado" >> /etc/issue
root@a1b2c3d4:/# exit

# 3. Auditar la capa de lectura/escritura desde el host
$ docker diff demo-cow
C /etc
C /etc/issue           # C = Changed (Copiado a R/W)
A /root/.bash_history  # A = Added

# 4. Comprobar que la imagen base original está intacta
$ docker run --rm ubuntu:22.04 cat /etc/issue
```

CICLO DE VIDA

Ciclo de Vida y Máquina de Estados del Contenedor

1

Created

Se prepara el sistema de archivos Overlay2 y la configuración de red, pero el proceso PID 1 aún no ha arrancado en el kernel.

2

Running

El proceso principal está activo consumiendo ciclos de CPU y RAM bajo sus respectivos namespaces y cgroups.

3

Paused

El kernel congela los procesos mediante el cgroup freezer sin liberar la memoria RAM asignada.

4

Stopped

El proceso finaliza tras recibir SIGTERM o SIGKILL. El sistema de archivos R/W persiste en disco hasta hacer `docker rm`.

Parada Limpia (SIGTERM · Señal 15)

- **Notificación de Cierre:** Docker avisa a la aplicación para que cierre conexiones a base de datos y vacíe buffers.
- **Periodo de Gracia:** 10 segundos por defecto (configurable con `--time` o directiva `STOPSIGNAL`).

Parada Forzosa (SIGKILL · Señal 9)

- **Terminación Inmediata:** El kernel aborta el proceso sin posibilidad de captura ni limpieza de recursos.
- **Mitigación con `--init`:** Inyectar un mini-init (`tini`) para propagar señales y cosechar procesos zombies.

OPERACIÓN CLI

Comandos Esenciales de Operación en el Aula

GESTIÓN DE CONTENEDORES

```
# Ejecutar en segundo plano con puerto y nombre
$ docker run -d --name mi-nginx -p 8080:80 nginx:alpine

# Listar contenedores activos y todos (incluidos parados)
$ docker ps
$ docker ps -a

# Ver logs en tiempo real (seguimiento continuo)
$ docker logs -f --tail 50 mi-nginx

# Ejecutar una terminal interactiva dentro del contenedor
$ docker exec -it mi-nginx sh

# Inspeccionar metadatos y dirección IP
$ docker inspect -f '{{.NetworkSettings.IPAddress}}' mi-nginx
```

GESTIÓN DE IMÁGENES Y LIMPIEZA

```
# Descargar una imagen desde Docker Hub
$ docker pull redis:7-alpine

# Listar imágenes locales y su tamaño en disco
$ docker images

# Detener y eliminar un contenedor
$ docker stop mi-nginx
$ docker rm mi-nginx

# Diagnóstico del espacio en disco ocupado
$ docker system df

# Limpieza profunda de recursos huérfanos
$ docker system prune -af --volumes
```

Automatizando la Creación: El Dockerfile

Infraestructura como Código (IaC)

- **Fichero Declarativo:** Instrucciones secuenciales legibles que describen paso a paso cómo construir la imagen.
- **Versionado en Git:** El Dockerfile vive junto al código fuente, garantizando auditoría y trazabilidad histórica.
- **Compilador BuildKit:** Motor moderno que analiza dependencias y ejecuta etapas independientes en paralelo.
- **Reproducibilidad Total:** Cualquier miembro del equipo compilará exactamente el mismo artefacto binario.

DOCKERFILE PROFESIONAL PARA NODE.JS

```
# 1. Imagen base oficial y minimalista
FROM node:20-alpine

# 2. Establecer directorio de trabajo seguro
WORKDIR /usr/src/app

# 3. Copiar dependencias y compilar (Caché optimizada)
COPY package*.json ./
RUN npm ci --only=production

# 4. Copiar código fuente de la aplicación
COPY . .

# 5. Ejecutar como usuario sin privilegios y documentar puerto
USER node
EXPOSE 3000

# 6. Comando de arranque en formato Exec
CMD ["node", "server.js"]
```

DIRECTIVAS DOCKERFILE

Directivas Fundamentales: Entorno, Ejecución y CMD vs ENTRYPOINT

Directivas de Entorno y Sistema

- **FROM <imagen>:<tag>:** Establece la imagen base obligatoria (primera instrucción).
- **WORKDIR /ruta:** Define el directorio de trabajo activo (evita encadenar múltiples `cd`).
- **COPY <src> <dest>:** Copia ficheros locales del host hacia la imagen de forma transparente.
- **ENV CLAVE=VALOR:** Define variables de entorno persistentes durante el build y en runtime.
- **EXPOSE <puerto>:** Metadato informativo que documenta los puertos que escucha la aplicación.

RUN vs ENTRYPOINT vs CMD

- **RUN (Momento Build):** Instala paquetes y compila código, creando una nueva capa inmutable en disco.
- **ENTRYPOINT ["ejecutable"]:** Define el comando fijo que siempre se ejecutará al iniciar el contenedor.
- **CMD ["arg1", "arg2"]:** Proporciona argumentos por defecto al ENTRYPOINT (anulables desde el CLI).
- **Sintaxis Exec Obligatoria:** Siempre usar formato JSON con corchetes `["prog", "arg"]` para ser PID 1.

Buenas Prácticas: Optimización de Capas y Caché de Build

Estrategias de Optimización

- **De Menos a Más Cambiante:** Ordenar dependencias del sistema primero y código fuente después.
- **Invalidación en Cascada:** Cualquier modificación en una capa invalida la caché de todas las subsiguientes.
- **Uso de `.dockerignore`:** Excluir `node_modules`, `.git` y temporales del build context de compilación.
- **Encadenar Comandos RUN:** Actualizar paquetes, instalar y limpiar cachés en una única capa atómica.

ANTIPATRÓN VS BUENA PRÁCTICA

```
# ANTIPATRÓN (Rompe caché en cada cambio de código):
COPY . .
RUN npm install   # ¡Se ejecuta siempre, muy lento!

# BUENA PRÁCTICA (Caché inteligente de librerías):
COPY package*.json ./
RUN npm ci        # Solo se ejecuta si cambia package.json
COPY . .          # Si solo cambia código, usa caché previa

# Ejemplo de limpieza de paquetes en una sola capa:
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    ca-certificates \
 && rm -rf /var/lib/apt/lists/*
```

MULTI-STAGE

Multi-Stage Builds: Seguridad y Mínimo Tamaño en Producción

Ventajas del Patrón Multi-Stage

- **Separación Compilación/Runtime:** SDKs pesados y compiladores se quedan exclusivamente en la fase de build.
- **Producción Ultraligera:** Imágenes finales de 5MB a 30MB en lugar de artefactos de más de 1GB.
- **Minimización de CVEs:** Al eliminar gestores de paquetes y shells del entorno final, la superficie se reduce a cero.
- **Builds Paralelos en BuildKit:** El motor compila etapas independientes de forma simultánea.

DOCKERFILE MULTI-STAGE PROFESIONAL (GOLANG)

```
# ETAPA 1: Entorno de Compilación (Builder)
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o mi-app

# ETAPA 2: Runtime de Producción (Final)
FROM alpine:3.19
WORKDIR /app
# Copiar ÚNICAMENTE el binario de la etapa anterior
COPY --from=builder /app/mi-app .
USER guest
EXPOSE 8080
CMD ["./mi-app"]
# Resultado: De 850MB a solo 12MB de imagen final
```

ARQUITECTURA

Docker y la Arquitectura de Microservicios: Un Proceso por Contenedor

La Regla de Oro: Un Proceso por Contenedor

- **Desacoplamiento Funcional:** No empaquetar servidor web, API y base de datos en el mismo contenedor.
- **Escalado Horizontal Selectivo:** Escalar únicamente los servicios con alta demanda sin duplicar el resto.
- **Diagnóstico Aislado:** Los logs (`docker logs`) reflejan exclusivamente la salida del servicio en cuestión.
- **Aislamiento de Fallos:** Un colapso en el módulo de pagos no detiene el catálogo de navegación.

Retos que Resolveremos en el Curso

- **Comunicación entre Servicios:** ¿Cómo hablan entre sí sin acoplar IPs fijas? (Módulo 2 · Redes Bridge y DNS).
- **Persistencia de Datos:** Si el contenedor es efímero, ¿dónde guardamos datos? (Módulo 3 · Volúmenes).
- **Orquestación Multicontenedor:** Coordinar 15 servicios con un solo comando (Módulo 4 · Docker Compose).
- **Producción y Alta Disponibilidad:** Despliegues en clúster y balanceo de carga (Módulo 5 · Kubernetes).

Laboratorio 1: Ciclo de Vida y Creación de Imágenes

Objetivos y Pasos del Laboratorio

- **Paso 1 · Verificación:** Comprobar el entorno con `docker version` y `docker info`.
- **Paso 2 · Exploración:** Ejecutar contenedor interactivo Alpine e inspeccionar procesos del kernel.
- **Paso 3 · Dockerfile:** Redactar una receta optimizada para una aplicación web con capas ordenadas.
- **Paso 4 · Build:** Compilar la imagen con etiqueta semántica inmutable `lab1-app:1.0`.
- **Paso 5 · Despliegue:** Instanciar el contenedor en segundo plano publicando el puerto hacia el host.
- **Paso 6 · Auditoría:** Comprobar conectividad con `curl`, monitorizar `docker stats` y `docker logs`.
- **Paso 7 · Limpieza:** Parar y eliminar el contenedor verificando la liberación de recursos.

COMANDOS DE RESOLUCIÓN DEL LABORATORIO 1

```
# 1. Crear espacio de trabajo y Dockerfile de prueba
$ mkdir -p ~/lab1 && cd ~/lab1
$ cat <<EOF > Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
EOF

# 2. Crear archivo HTML simple
$ echo "<h1>Laboratorio 1 Completado</h1>" > index.html

# 3. Compilar la imagen con etiqueta semántica
$ docker build -t lab1-app:1.0 .

# 4. Arrancar contenedor en segundo plano con puerto
$ docker run -d --name lab1-web -p 8080:80 lab1-app:1.0

# 5. Probar conectividad y revisar logs
$ curl -I http://localhost:8080
$ docker logs lab1-web

# 6. Limpieza final
$ docker rm -f lab1-web
```

01

MÓDULO 1 · CONCLUSIÓN DÍA 1

RESUMEN DE LA JORNADA 1

- Kernel de Linux: Namespaces (visibilidad) y Cgroups (recursos).
- Arquitectura OCI: Desacoplamiento dockerd, containerd y runc.
- Imágenes y Almacenamiento: Capas inmutables y Copy-on-Write en Overlay2.
- Dockerfile Profesional: BuildKit, optimización de caché y Multi-Stage.
- Laboratorio 1 completado con éxito por los alumnos.

Conceptos consolidados: Arquitectura del Motor, Namespaces, Cgroups, OCI, Dockerfile y Ciclo de Vida.
Próxima Jornada: Módulo 2 (Redes a fondo) y Módulo 3 (Almacenamiento y Volúmenes).

02

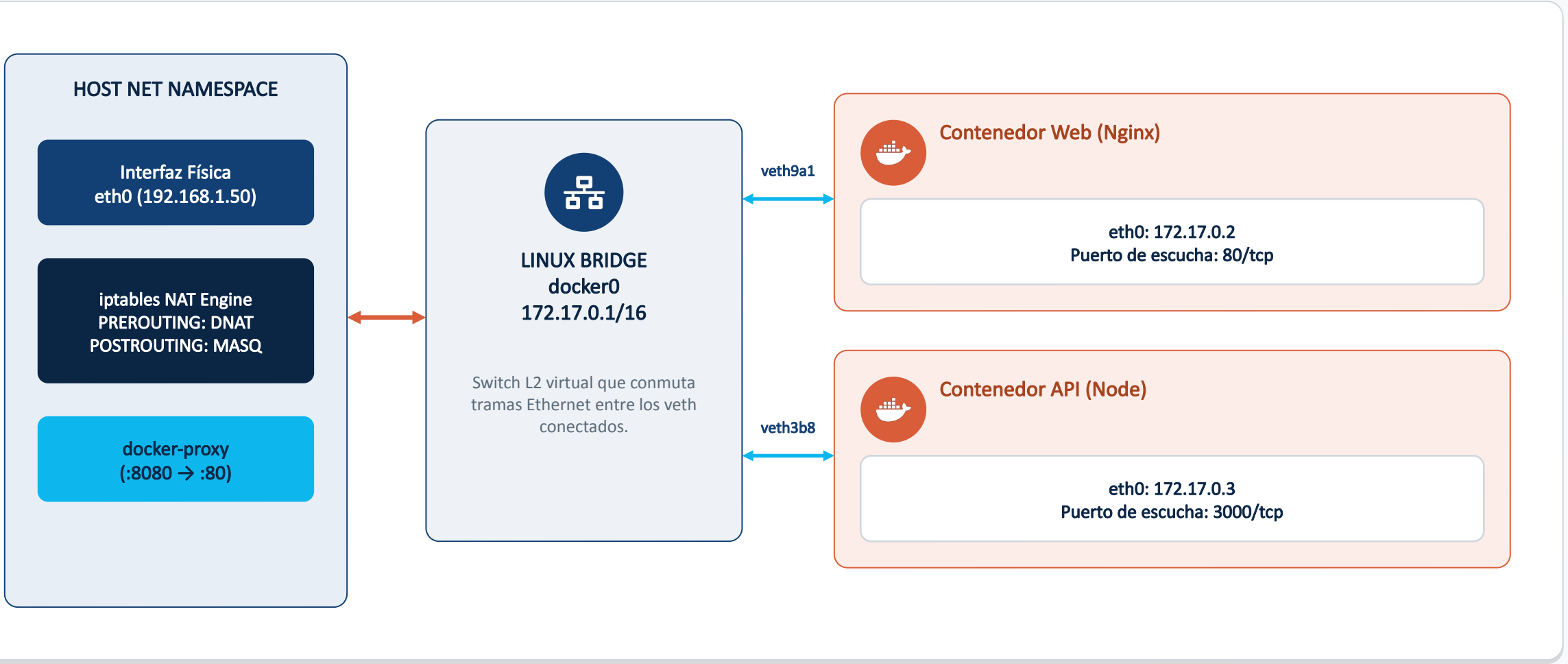
MÓDULO 2 · 2.5 HORAS LECTIVAS

GESTIÓN DE REDES EN DOCKER

Arquitectura de Red: veth pairs, Linux Bridge, DNS interno, iptables NAT, Host, None, Overlay y Macvlan.
Primera mitad de la Jornada 2 · 2.5 Horas Lectivas

- Anatomía de Red: veth pairs, switch virtual docker0 y namespaces.
- Bridge por defecto vs Redes personalizadas (User-Defined DNS).
- Mapeo de puertos, reglas iptables DNAT y docker-proxy.
- Drivers avanzados: Host (rendimiento), None (offline) y Macvlan (LAN física).
- Laboratorio 2: Interconexión de microservicios y resolución por nombre.

Anatomía de la Red en Docker: veth pairs, Linux Bridge e iptables



La Red Bridge por Defecto y sus Limitaciones Críticas

Características de la Red Bridge por Defecto

- **Conexión Automática:** Red asignada si se omite el parámetro `--network` en `docker run`.
- **Rango de Subred:** Típicamente `172.17.0.0/16`, con la IP `172.17.0.1` asignada al host como gateway.
- **Salida a Internet (NAT):** Docker inyecta reglas `iptables MASQUERADE` para permitir tráfico saliente.
- **Asignación Dinámica:** IPs asignadas de forma secuencial según el orden de arranque de los contenedores.

Limitación Crítica: Sin Resolución DNS

- **¡Sin DNS Integrado!:** Dos contenedores en el bridge por defecto NO pueden comunicarse por su nombre.
- **Fragilidad por IPs:** Si un servicio se reinicia y cambia de `172.17.0.2` a `172.17.0.3`, la app colapsa.
- **Deprecación de `--link`:** El antiguo mecanismo de inyección en `/etc/hosts` está formalmente en desuso.
- **Solución Profesional:** Crear siempre redes personalizadas (*User-Defined Bridges*).

TRAMPA EN PRODUCCIÓN: Nunca desplegar microservicios en la red bridge predeterminada. La falta de DNS obliga a acoplar direcciones IP volátiles que se rompen con cada reinicio o actualización de contenedor.

Redes Personalizadas: Resolución DNS Automática y Aislamiento

Ventajas de User-Defined Bridges

- **DNS Embebido (`127.0.0.11`):** Resolución automática y dinámica del nombre del contenedor a su IP activa.
- **Aislamiento Estricto:** Servicios en redes diferentes no tienen visibilidad mutua, reforzando la seguridad.
- **Conexión en Caliente:** Conectar o desconectar contenedores en tiempo de ejecución sin reinicios (`docker network connect`).
- **Alias de Red:** Asignar nombres secundarios a servicios para balanceo o nombres canónicos (`--network-alias`).

CREACIÓN Y RESOLUCIÓN EN RED PERSONALIZADA

```
# 1. Crear red personalizada para el backend
$ docker network create red-aplicacion

# 2. Levantar base de datos conectada a la red
$ docker run -d --name db-postgres \
    --network red-aplicacion \
    -e POSTGRES_PASSWORD=secreto \
    postgres:16-alpine

# 3. Levantar cliente en la misma red y resolver por nombre
$ docker run --rm --network red-aplicacion alpine \
    ping -c 2 db-postgres
PING db-postgres (172.19.0.2): 56 data bytes
64 bytes from 172.19.0.2: seq=0 ttl=64 time=0.075 ms

# ¡El DNS embebido resuelve por nombre sin tocar /etc/hosts!
```

PORT MAPPING

Mapeo de Puertos: Abriendo el Contenedor al Mundo

Mapeo Explícito vs Efímero

- **Mapeo Explícito (`-p`):** Vincula un puerto fijo del host al contenedor (ej. `-p 8080:80` o `-p 127.0.0.1:8080:80`).
- **Mapeo Efímero (`-P`):** Asigna puertos aleatorios altos (32768-60999) para todas las directivas `EXPOSE`.
- **Riesgo de Exposición (`0.0.0.0`):** Omitir la IP local expone el puerto a cualquier red física o Internet.
- **Mapeo UDP:** Por defecto mapea TCP. Para tráfico UDP se debe indicar `-p 53:53/udp`.

MECANISMO INTERNO: IPTABLES DNAT

```
# Inspeccionar las reglas generadas en la tabla NAT
$ sudo iptables -t nat -L DOCKER -n -v

Chain DOCKER (2 references)
pkts bytes target prot opt source   destination
  22  1320 DNAT   tcp  --  0.0.0.0  0.0.0.0 \
  tcp dpt:8080 to:172.19.0.2:80

# Proceso docker-proxy (Reenvío TCP en loopback):
$ ps -ef | grep docker-proxy
root ... docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8080
```

AVISO DE SEGURIDAD: En servidores de desarrollo, usar `-p 127.0.0.1:puerto:puerto`. El uso desatento de `-p puerto:puerto` abre el servicio a toda la red corporativa o pública sin pasar por UFW.

MODO HOST

Modo de Red Host: Rendimiento Máximo sin Aislamiento

Características de `--network host`

- **Sin Namespace de Red:** El proceso utiliza directamente la pila TCP/IP, interfaces y rutas del host.
- **Cero Overhead de NAT:** Sin pares veth, sin conmutador bridge y sin consumo de CPU en iptables.
- **Mapeo Innecesario:** Los flags `-p` y `-P` se ignoran; el servicio abre puertos directamente en el host.
- **Comportamiento en Desktop:** En Mac/Windows se asocia a la máquina virtual oculta, no a macOS/Windows real.

Riesgos y Casos de Uso Válidos

- **Colisión de Puertos:** Dos contenedores no pueden escuchar en el mismo puerto (`Address already in use`).
- **Pérdida de Aislamiento:** El contenedor tiene acceso a todos los servicios que escuchen en `localhost` del host.
- **Monitorización de Red:** Ideal para agentes de métricas (Prometheus `node-exporter`, capturadores de tráfico).
- **Tráfico de Alta Velocidad:** Transmisión multimedia UDP/WebRTC o proxies de extrema concurrencia.

MODO NONE

Modo de Red None: Aislamiento Total y Entornos Air-Gapped



Aislamiento Estricto (`--network none`)

- **Desconexión Absoluta:** Únicamente dispone de la interfaz de bucle invertido local (`lo` / `127.0.0.1`).
- **Sin Tráfico Externo:** Imposible enviar o recibir tramas fuera del contenedor; sin ruta por defecto.
- **Inmunidad frente a Exfiltración:** Código potencialmente comprometido no puede conectar con servidores externos C2.
- **Configuración Manual:** Permite a orquestadores inyectar interfaces de red personalizadas a posteriori.



Casos de Uso en Producción

- **Procesamiento Confidencial:** Transformación de expedientes médicos, nóminas o registros protegidos por GDPR.
- **Operaciones Criptográficas:** Generación de llaves maestras PGP, firma de artefactos o monederos fríos.
- **Sandboxing de Código:** Ejecución aislada de scripts no confiables subidos por clientes sin riesgo de propagación.
- **Inspección Forense:** Análisis estático de binarios sospechosos en entornos cerrados (air-gapped).

REDES OVERLAY

Redes Overlay: Conectando Contenedores a Través de Múltiples Hosts

Encapsulación VXLAN (L2 sobre L3)

- **Reto Multi-Host:** Comunicar contenedores ubicados en diferentes servidores físicos sin ruteo manual.
- **Túneles VXLAN:** Encapsula tramas Ethernet L2 en paquetes UDP estándar (puerto 4789) sobre la red física.
- **Cifrado IPsec Nativo:** Soporte de cifrado en tránsito transparente con la opción `--opt encrypted`.
- **Descubrimiento Distribuido:** El plan de control sincroniza las IPs y nombres entre todos los nodos del clúster.

Ámbitos de Aplicación y Orquestación

- **Docker Swarm:** Driver nativo por defecto para interconectar réplicas de servicios distribuidos.
- **Kubernetes (CNI):** Base conceptual de plugins de red como Calico, Flannel o Cilium (con eBPF).
- **Independencia Física:** Los servicios se ven en una subred privada virtual común (ej. `10.0.0.0/24`).
- **Control Plane Distribuido:** Utiliza protocolos gossip y consenso Raft para propagar rutas sin cuello de botella.

Macvlan y Complementos de Red: Integración en LAN Física

Driver Macvlan (IP en la LAN Física)

- **Dirección MAC Propia:** Cada contenedor tiene su propia MAC y solicita IP directa al router/DHCP de la LAN.
- **Sin NAT ni Port Mapping:** Aparece en la red física como un equipo más conectado directamente al switch.
- **Migración Legacy:** Ideal para software antiguo que rechaza NAT o requiere puertos estándar sin mapeo.
- **Modo Promiscuo:** La tarjeta física del host debe soportar modo promiscuo para cursar tráfico multi-MAC.

Complementos de Red y Plugins CNI

- **Ecosistema Extensible:** Instalación de controladores de terceros mediante `docker plugin install`.
- **Calico / Cilium:** Añaden políticas de seguridad por software (Network Policies) y filtrado eBPF.
- **Cifrado WireGuard:** Encriptación punto a punto de todo el tráfico entre contenedores sin configuración manual.
- **Enrutamiento BGP:** Propagación directa de rutas de contenedores hacia los routers troncales del datacenter.

Laboratorio 2: Redes Personalizadas y DNS Embebido

Objetivos del Laboratorio 2

- **Paso 1 · Inspección:** Analizar la red bridge por defecto y constatar la ausencia de DNS interno.
- **Paso 2 · Red Propia:** Crear `mi-red-app` utilizando el driver estándar bridge.
- **Paso 3 · Servicio Cache:** Desplegar un contenedor Redis asociado directamente a la red creada.
- **Paso 4 · Verificación DNS:** Invocar un contenedor cliente y hacer ping al hostname `cache-redis`.
- **Paso 5 · Aislamiento:** Comprobar que contenedores en redes distintas no pueden comunicarse.
- **Paso 6 · En Caliente:** Vincular un contenedor a dos redes a la vez con `docker network connect`.
- **Paso 7 · Limpieza:** Eliminar los contenedores y destruir las redes creadas.

RESOLUCIÓN DEL LABORATORIO 2

```
# 1. Crear red de trabajo personalizada
$ docker network create --driver bridge mi-red-app

# 2. Desplegar Redis en segundo plano
$ docker run -d --name cache-redis \
    --network mi-red-app redis:alpine

# 3. Probar resolución DNS embebida por nombre
$ docker run --rm --network mi-red-app alpine \
    ping -c 2 cache-redis

# 4. Crear red independiente de auditoría
$ docker network create red-auditoria
$ docker run -d --name app-aislada \
    --network red-auditoria alpine sleep 3600

# 5. Conectar en caliente el contenedor a la primera red
$ docker network connect mi-red-app app-aislada
$ docker exec app-aislada ping -c 2 cache-redis

# 6. Limpieza ordenada
$ docker rm -f cache-redis app-aislada
$ docker network rm mi-red-app red-auditoria
```

03

MÓDULO 3 · 2.5 HORAS LECTIVAS

ALMACENAMIENTO Y VOLÚMENES

Persistencia de Datos: Storage Drivers (Overlay2), Named Volumes, Bind Mounts, Tmpfs, Backups y Migraciones.
Segunda mitad de la Jornada 2 · 2.5 Horas Lectivas

- Efímero vs Persistente: Desacoplar almacenamiento de cómputo.
- Overlay2 a fondo: lowerdir, upperdir, merged y ficheros whiteout.
- Los 3 mecanismos: Named Volumes, Bind Mounts y Tmpfs en RAM.
- Estrategias de Backup, Restauración y Migración de Volúmenes.
- Laboratorio 3: Persistencia de BD y Hot-Reloading en desarrollo.

PERSISTENCIA

El Reto de los Datos: Naturaleza Efímera vs Persistencia Crítica



Naturaleza Efímera del Contenedor

- **Contenedores Desechables:** Deben poder destruirse, actualizarse y recrearse sin temor a perder información.
- **Sobrecarga de Copy-on-Write:** La capa R/W añade penalizaciones de rendimiento en escrituras aleatorias masivas.
- **Pérdida Irrecuperable:** Ejecutar `docker rm -f` destruye la capa superior y sus datos para siempre.
- **Dificultad de Backup:** Extraer copias de seguridad desde capas internas del driver es complejo y propenso a fallos.



Solución: Desacoplamiento de Datos

- **Cómputo vs Almacenamiento:** Los datos residen en estructuras de almacenamiento ajenas al ciclo del contenedor.
- **Rendimiento Nativo de E/S:** Los volúmenes omiten Overlay2 y escriben a la velocidad de disco nativa del host.
- **Compartición Concurrente:** Múltiples contenedores pueden montar el mismo volumen simultáneamente.
- **Persistencia Garantizada:** Si el contenedor colapsa o se borra, los datos permanecen completamente a salvo.

Overlay2 a Fondo: La Magia del Union File System

Los 4 Directorios de Overlay2

- **lowerdir:** Colección de capas de solo lectura apiladas provenientes de la imagen OCI.
- **upperdir:** Directorio de lectura/escritura exclusivo del contenedor para modificaciones y nuevos ficheros.
- **workdir:** Directorio interno de trabajo del kernel para preparar copias atómicas antes del commit.
- **merged:** Punto de montaje `/` unificado que percibe el proceso del contenedor.
- **Ficheros Whiteout:** Dispositivos de caracteres (0,0) creados en el upperdir para ocultar archivos borrados.

INSPECCIÓN DEL GRAPHDRIVER EN DISCO

```
# Inspeccionar los directorios asignados por Overlay2
$ docker inspect -f '{{json .GraphDriver.Data}}' mi-app
{
  "LowerDir": "/var/lib/docker/overlay2/abc.../diff:...",
  "MergedDir": "/var/lib/docker/overlay2/def.../merged",
  "UpperDir": "/var/lib/docker/overlay2/def.../diff",
  "WorkDir": "/var/lib/docker/overlay2/def.../work"
}

# ¿Qué ocurre al borrar un fichero de la imagen?
# Overlay2 genera un whiteout en Upperdir que oculta
# el inodo de Lowerdir sin liberar el espacio en disco.
$ ls -l /var/lib/docker/overlay2/def.../diff/fichero_borrado
c--------- 1 root root 0, 0 oct 27 10:00 fichero_borrado
```

Los Tres Mecanismos de Persistencia: Volumes, Binds y Tmpfs

Named Volumes

- **Ubicación:** `/var/lib/docker/volumes/`.
- **Gestión:** Administrado íntegramente por el motor Docker.
- **Seguridad:** Aislado de manipulaciones indebidas en el host.
- **Soporte Cloud:** Compatible con plugins de red (NFS, AWS EBS).
- **Uso Canónico:** Bases de datos y almacenamiento en producción.

Bind Mounts

- **Ubicación:** Cualquier directorio o archivo del host (`/ruta`).
- **Gestión:** Dependiente de la estructura de ficheros del host.
- **Edición en Vivo:** El host y el contenedor comparten acceso simultáneo.
- **Riesgos:** Desalineación de permisos numéricos UID/GID.
- **Uso Canónico:** Desarrollo local con recarga en caliente (hot-reload).

Tmpfs Mounts

- **Ubicación:** Memoria RAM del sistema anfitrión.
- **Gestión:** No escribe jamás bloques en disco físico.
- **Velocidad:** Rendimiento de E/S máximo a velocidad de memoria.
- **Volatilidad:** Se destruye totalmente al parar el contenedor.
- **Uso Canónico:** Secretos temporales, tokens OAuth y cachés volátiles.

NAMED VOLUMES

Volúmenes con Nombre (Named Volumes): Estándar de Producción

Ciclo de Vida de los Volúmenes

- **Independencia Total:** Los volúmenes sobreviven al borrado de cualquier contenedor que los monte.
- **Poblado Automático:** Docker copia los archivos preexistentes de la imagen hacia el volumen nuevo en el primer arranque.
- **Sintaxis Explícita `--mount`:** Mayor legibilidad y diferenciación entre volúmenes, binds y tmpfs.
- **Higiene del Sistema:** Comando `docker volume prune` para liberar volúmenes que han quedado huérfanos.

DEMOSTRACIÓN DE PERSISTENCIA REAL

```
# 1. Crear volumen explícito para PostgreSQL
$ docker volume create pgdata

# 2. Desplegar BD montando el volumen con sintaxis moderna
$ docker run -d --name db-postgres \
    --mount source=pgdata,target=/var/lib/postgresql/data \
    -e POSTGRES_PASSWORD=secreto postgres:16-alpine

# 3. Destruir forzosamente el contenedor
$ docker rm -f db-postgres

# 4. Recrear el contenedor a partir de una versión superior
$ docker run -d --name db-postgres-nuevo \
    --mount source=pgdata,target=/var/lib/postgresql/data \
    postgres:16-alpine

# ¡Todas las tablas y datos continúan intactos en pgdata!
```

Bind Mounts: Desarrollo Ágil y Riesgos de Permisos UID/GID

La Herramienta del Desarrollador

- **Edición en Vivo (Hot-Reload):** Modificar código en el IDE del host y comprobar cambios sin reconstruir la imagen.
- **Rutas Absolutas Obligatorias:** Requiere indicar rutas completas del host (`$(pwd)/src:/app/src`).
- **Riesgo de Ocultación:** Montar un directorio vacío oculta los archivos preexistentes en esa ruta del contenedor.
- **Modo Solo Lectura (`:ro`):** Buena práctica para proteger configuraciones críticas y evitar escrituras no deseadas.

USO SEGURO Y RESOLUCIÓN DE PERMISOS

```
# 1. Montaje para desarrollo web con recarga en caliente
$ docker run -d --name web-dev \
    -p 3000:3000 \
    -v "$(pwd)/src:/app/src:ro" \
    node:20-alpine npm run dev

# 2. Solución al problema de permisos UID/GID:
# Si el contenedor corre como UID 0, los archivos creados
# pertenecerán a root en el host. Pasar el UID propio:
$ docker run --user "$(id -u):$(id -g)" \
    -v "$(pwd):/workspace" mi-compilador make

# 3. Sintaxis explícita con --mount en solo lectura:
$ docker run --mount type=bind,source=/etc/ssl,target=/certs,readonly \
    nginx
```

Tmpfs Mounts: Almacenamiento en Memoria RAM para Secretos

Almacenamiento Volátil de Alta Seguridad

- **Directo a Memoria RAM:** Los archivos se escriben en memoria sin tocar jamás los discos SSD/NVMe del servidor.
- **Cero Huella Forense:** Al parar el contenedor los datos desaparecen sin dejar fragmentos en disco.
- **Custodia de Secretos:** Óptimo para certificados efímeros, llaves privadas y tokens de autenticación.
- **Límites Estrictos:** Configurar `tmpfs-size` para evitar que un contenedor desbocado agote la memoria del host.

MONTAJE TMPFS EN TERMINAL

```
# 1. Montar directorio seguro en RAM con límite de 64MB
$ docker run -d --name app-segura \
    --tmpfs /app/secrets:rw,noexec,nosuid,size=64m \
    nginx:alpine

# 2. Comprobar que está respaldado por memoria
$ docker exec app-segura df -h /app/secrets
Filesystem  Size  Used  Avail  Use%  Mounted on
tmpfs       64M      0    64M    0%  /app/secrets

# 3. Sintaxis formal mediante --mount:
$ docker run --mount type=tmpfs,destination=/app/cache,\
    tmpfs-size=128m,tmpfs-mode=1777 mi-app
```

PERSPECTIVA HISTÓRICA

Contenedores de Volúmenes de Datos: Perspectiva y Desuso

El Patrón `--volumes-from` (2014)

- **El Problema en 2014:** En versiones tempranas de Docker no existían comandos de gestión para volúmenes.
- **Contenedor Centinela:** Se creaba un contenedor tonto que definía volúmenes y permanecía detenido (`Exited`).
- **Uso de `--volumes-from`:** Otros contenedores montaban sus datos haciendo referencia al contenedor detenido.
- **Fragilidad:** Si alguien ejecutaba `docker rm -v` sobre el centinela, los datos se eliminaban sin aviso.

Por Qué Hoy Está en Desuso

- **Llegada de Docker 1.9:** Se introdujo el subsistema nativo completo `docker volume`.
- **Ciudadanos de Primera Clase:** Los volúmenes tienen nombre, ciclo de vida e inventario propio e independiente.
- **Drivers y Plugins Modernos:** Permiten conectarse a almacenamiento en red (NFS, AWS EBS), imposible con el patrón viejo.
- **Conclusión:** Comprenderlo para código legacy, pero utilizar exclusivamente Named Volumes en proyectos nuevos.

Drivers de Almacenamiento Remoto: NFS, SAN y Cloud

Almacenamiento Distribuido y de Red

- **Más Allá del Disco Local:** Permite que los datos sigan al contenedor si se desplaza entre diferentes nodos.
- **Driver Local con Opciones:** El driver `local` puede invocar llamadas `mount` para NFS, CIFS o SSHFS.
- **Integración Cloud:** Plugins especializados para montar discos elásticos en AWS, Azure y Google Cloud.
- **Alta Disponibilidad:** Permite desacoplar el ciclo de vida del servidor físico del almacenamiento de datos.

MONTAJE DE VOLUMEN NFS EN DOCKER

```
# 1. Crear volumen Docker respaldado por un servidor NFS
$ docker volume create --driver local \
    --opt type=nfs \
    --opt o=addr=192.168.1.100,rw,noatime,rsize=8192 \
    --opt device=:/srv/nfs/datos_compartidos \
    volumen-nfs

# 2. Montar el volumen en cualquier contenedor
$ docker run -d --name app-nfs \
    -v volumen-nfs:/app/storage \
    mi-aplicacion:1.0

# Si el contenedor se traslada a otro servidor físico,
# montará exactamente el mismo volumen NFS con los mismos datos.
```

Estrategias de Backup, Restauración y Migración de Volúmenes

El Patrón Contenedor de Backup

- **Sin Parar el Motor:** No requiere acceso root directo a `/var/lib/docker` ni reiniciar servicios.
- **Contenedor Efímero (`--rm`):** Un contenedor Alpine monta el volumen en solo lectura y la carpeta de backups.
- **Automatizable en Cron:** Se ejecuta en una sola línea de terminal ideal para tareas programadas de madrugada.
- **Migración entre Servidores:** El archivo `.tar.gz` resultante puede enviarse por scp o rsync a otro host y restaurarse.

COMANDOS DE BACKUP Y RESTAURACIÓN

```
# 1. BACKUP: Empaquetar el volumen "pgdata" en un tar comprimido
$ docker run --rm \
    -v pgdata:/volumen_origen:ro \
    -v $(pwd)/backups:/backup_destino \
    alpine tar czf /backup_destino/pgdata_$(date +%F).tar.gz \
           -C /volumen_origen .

# 2. RESTAURACIÓN: Descomprimir en un volumen nuevo
$ docker volume create pgdata_recuperado
$ docker run --rm \
    -v pgdata_recuperado:/volumen_destino \
    -v $(pwd)/backups:/backup_destino \
    alpine tar xzf /backup_destino/pgdata_2026-10-27.tar.gz \
           -C /volumen_destino
```

BUENAS PRÁCTICAS

Buenas Prácticas en Gestión de Persistencia y Antipatrones

Reglas de Oro en Producción

- **Datos Fuera del Contenedor:** Toda información persistente debe residir en Named Volumes o Bind Mounts.
- **Nombres Descriptivos:** Nombrar siempre los volúmenes explícitamente para evitar volúmenes anónimos.
- **Partición Separada en Host:** Montar `/var/lib/docker` en un disco o LVM dedicado para no saturar `/` del host.
- **Auditoría con Labels:** Etiquetar volúmenes (`--label env=prod`) para facilitar políticas de retención.

Antipatrones a Erradicar

- **Antipatrón `docker commit`:** Hacer commits manuales para guardar estado destruye la reproducibilidad del Dockerfile.
- **Rutas Absolutas de Portátil:** Subir rutas de tipo `/Users/juan/...` a producción rompe el despliegue.
- **Descuido de Permisos:** Olvidar comprobar si el usuario no root del contenedor tiene permisos de escritura en el volumen.
- **Volúmenes Huérfanos Acumulados:** No realizar mantenimiento de `docker volume prune`, saturando el almacenamiento.

Laboratorio 3: Persistencia de BD y Hot-Reloading en Desarrollo

Objetivos del Laboratorio 3

- **Parte A · Persistencia BD:** Crear Named Volume `db_datos` y arrancar PostgreSQL.
- **Paso 2 · Inserción:** Crear una tabla de usuarios e insertar registros de prueba con psql.
- **Paso 3 · Desastre Simulado:** Eliminar el contenedor forzosamente con `docker rm -f`.
- **Paso 4 · Comprobación:** Levantar un nuevo contenedor y verificar que las tablas siguen íntegras.
- **Parte B · Hot-Reloading:** Montar código del host en Nginx con Bind Mount en solo lectura.
- **Paso 6 · Edición en Vivo:** Editar el código en el host y comprobar la actualización instantánea sin rebuild.

RESOLUCIÓN DEL LABORATORIO 3

```
# PARTE A: Persistencia con Named Volume
$ docker volume create db_datos
$ docker run -d --name db-test -v db_datos:/var/lib/postgresql/data \
    -e POSTGRES_PASSWORD=secreto postgres:16-alpine
$ docker exec -i db-test psql -U postgres -c \
    "CREATE TABLE alumnos (id int, nombre text); \
     INSERT INTO alumnos VALUES (1, 'Ana');"
$ docker rm -f db-test
$ docker run -d --name db-recuperada -v db_datos:/var/lib/postgresql/data \
    -e POSTGRES_PASSWORD=secreto postgres:16-alpine
$ docker exec -i db-recuperada psql -U postgres -c \
    "SELECT * FROM alumnos;"

# PARTE B: Desarrollo Ágil con Bind Mount
$ mkdir -p ~/web-dev && echo "<h1>V 1.0</h1>" > ~/web-dev/index.html
$ docker run -d --name dev-nginx -p 8080:80 \
    -v ~/web-dev:/usr/share/nginx/html:ro nginx:alpine
$ curl http://localhost:8080
$ echo "<h1>V 2.0 (En Vivo)</h1>" > ~/web-dev/index.html
# ¡Cambio instantáneo en el navegador!
$ curl http://localhost:8080
```

03

MÓDULO 3 · CONCLUSIÓN DÍA 2

RESUMEN DE LA JORNADA 2

- Redes: El DNS embebido 127.0.0.11 hace obligatorias las redes personalizadas.
- Mapeo de Puertos: iptables DNAT y docker-proxy para acceso exterior.
- Persistencia: Desacoplar almacenamiento de cómputo para datos de producción.
- Named Volumes vs Bind Mounts vs Tmpfs: Casos de uso y directrices claras.
- Laboratorios 2 y 3 completados con éxito por los alumnos.

Conceptos consolidados: Redes (Bridge, Host, None, Overlay, DNS interno, iptables) y Persistencia (Overlay2, Volumes, Binds, Tmpfs).
Próxima Jornada: Módulo 4 (Contenedores en Cloud y Orquestación con Docker Compose).

04

MÓDULO 4 · 5 HORAS LECTIVAS

CONTENEDORES EN LA NUBE

Módulo 4: Containers en Azure, AWS, Google Cloud y Evolución de Docker Cloud
Jornada 3 de 4 · 5 Horas Lectivas (20h totales)

- Evolución Cloud: De IaaS a CaaS y Serverless Containers.
- Pipeline de despliegue: Build, Ship (Registries OCI) y Run.
- Microsoft Azure: ACR, Container Instances (ACI) y Apps (ACA).
- Amazon Web Services: ECR, ECS y cómputo Serverless con Fargate.
- Google Cloud: Artifact Registry, Cloud Run y sandbox gVisor.
- Laboratorio 4: Publicación en Registry y despliegue CaaS.

Evolución Cloud: De IaaS a CaaS y Serverless Containers

IaaS (VMs Tradicionales)

- **Gestión Completa:** Parchear el SO, gestionar kernels, configurar firewalls y agentes de seguridad.
- **Arranque Lento:** Escalar levantando nuevas máquinas virtuales tarda varios minutos.
- **Pago por Capacidad:** Facturación continua 24/7 aunque la CPU esté al 1% de uso.
- **Uso Predilecto:** Monolitos heredados o aplicaciones con control estricto de drivers hardware.

CaaS (Containers as a Service)

- **Control del Contenedor:** El desarrollador entrega la imagen Docker; el cloud gestiona el runtime.
- **Orquestación Gestionada:** El proveedor maneja ciclo de vida, balanceo de carga y reinicios automáticos.
- **Escalado Elástico:** Adición de réplicas en segundos respondiendo a la demanda de tráfico.
- **Ejemplos Clave:** Amazon ECS, Azure Container Apps, Google Cloud Run.

Serverless Containers

- **Cero Servidores:** Sin nodos que administrar, sin actualizaciones de clúster ni capacidad ociosa.
- **Escalado a Cero:** Sin tráfico activo, las réplicas bajan a 0 y el coste económico es exactamente 0€.
- **Pago por Consumo Exacto:** Facturación al milisegundo por vCPU y memoria RAM consumidas.
- **Uso Predilecto:** APIs REST, microservicios orientados a eventos y tareas batch puntuales.

PIPELINE CLOUD

El Ciclo de Vida Cloud: Build, Ship & Run

El Flujo Canónico del Contenedor

- **1. BUILD (Compilación):** El desarrollador o el servidor de CI/CD (GitHub Actions, GitLab) genera la imagen.
- **2. TAG (Versionado):** Se etiqueta con la URL del registry privado y el tag semántico (`acr.io/app:1.2.0`).
- **3. SHIP (Publicación):** Se sube al Container Registry mediante protocolos HTTPS autenticados con tokens.
- **4. RUN (Despliegue):** El servicio cloud descarga la imagen desde el registry privado y levanta el servicio.

COMANDOS DEL FLUJO CLOUD EN TERMINAL

```
# 1. Autenticación segura en el registro cloud
$ az acr login --name miregistry          # Azure
$ aws ecr get-login-password | docker... # AWS
$ gcloud auth configure-docker            # GCP

# 2. Etiquetar la imagen local con la ruta completa
$ docker tag mi-app:latest \
    miregistry.azurecr.io/servicios/mi-app:1.2.0

# 3. Enviar las capas diferenciales al registro remoto
$ docker push miregistry.azurecr.io/servicios/mi-app:1.2.0

# 4. Desplegar en servicio serverless gestionado
$ az container create \
    --resource-group rg-prod \
    --name mi-api \
    --image miregistry.azurecr.io/servicios/mi-app:1.2.0
```

REGISTRIES OCI

Container Registries: Seguridad, Escaneo de Vulnerabilidades y Gobernanza

Docker Hub vs Registries Cloud Gestionados

- **Docker Hub Público:** Catálogo de imágenes oficiales de la comunidad, con limitaciones de rate-limit anónimo.
- **Registries Privados Cloud:** ACR, ECR y Artifact Registry dentro de la red privada del proveedor de nube.
- **Autenticación IAM Nativa:** Acceso por roles y Service Accounts (elimina contraseñas en texto plano).
- **Geo-Replicación Global:** Imágenes sincronizadas entre datacenters para descargas en milisegundos.

Seguridad y Escaneo de Vulnerabilidades

- **Escaneo de CVEs en Push:** Análisis estático de paquetes del SO (apt, apk) alertando sobre fallos conocidos.
- **Firma Digital (Cosign/Notary):** Garantía criptográfica de que la imagen fue generada por el pipeline oficial.
- **Inmutabilidad de Tags:** Impedir la sobrescritura accidental o fraudulenta de versiones publicadas.
- **Políticas de Ciclo de Vida:** Eliminación programada de imágenes huérfanas tras 30 días para reducir costes.

POLÍTICA CORPORATIVA: Habilitar siempre la directiva "Immutable Tags" en el Container Registry empresarial. Evita que un build fallido o un atacante modifique una imagen ya validada en producción.

MICROSOFT AZURE

Contenedores en Microsoft Azure: Panorámica de Servicios

Azure Container Registry (ACR)

Registro privado OCI con soporte de geo-replicación, cifrado en reposo, escaneo con Microsoft Defender for Cloud y control RBAC mediante Entra ID.

Azure Container Instances (ACI)

La solución más directa para arrancar contenedores en segundos sin gestionar clústeres. Facturación por segundos y aislamiento por hipervisor.

Azure Container Apps (ACA)

Plataforma serverless construida sobre Kubernetes y KEDA. Escalado automático por peticiones HTTP o colas, soporte de microservicios y escalado a cero.

Azure App Service Containers

PaaS enfocado a aplicaciones web HTTP/HTTPS con dominios personalizados, certificados SSL gestionados, slots de staging y autenticación social integrada.

Azure Kubernetes Service (AKS)

Plataforma de Kubernetes administrada de nivel empresarial con integración en Azure CNI, nodos virtuales y gobierno corporativo avanzado.

Criterio de Elección

- Tareas batch / Jobs: ACI
- Microservicios / APIs: ACA
- Webapps tradicionales: App Service
- Clústeres complejos: AKS

Azure Container Instances (ACI): Contenedores sin Servidores

Características Principales de ACI

- **Arranque Inmediato:** Puesta en marcha en segundos sin aprovisionamiento de VMs.
- **Facturación por Segundos:** Pago exacto por el tiempo de ejecución y recursos (vCPU/RAM).
- **Aislamiento por Hipervisor:** Seguridad multinquilino estricta; no comparte kernel con otros clientes.
- **IP Pública y FQDN:** Generación automática de nombre DNS público accesible desde Internet.
- **Montaje de Volúmenes:** Soporte nativo para carpetas compartidas de Azure Files mediante CIFS.

DESPLIEGUE EN ACI MEDIANTE AZURE CLI

```
# 1. Desplegar contenedor web con FQDN público
$ az container create \
    --resource-group rg-formacion \
    --name mi-web-aci \
    --image mcr.microsoft.com/azuredocs/aci-helloworld \
    --dns-name-label curso-docker-demo \
    --ports 80 \
    --cpu 1 \
    --memory 1.5

# 2. Consultar el FQDN asignado
$ az container show --resource-group rg-formacion \
    --name mi-web-aci --query "ipAddress.fqdn"
"curso-docker-demo.eastus.azurecontainer.io"

# 3. Eliminar la instancia al terminar la tarea
$ az container delete --resource-group rg-formacion \
    --name mi-web-aci -y
```

AZURE ACA

Azure Container Apps (ACA): Microservicios y KEDA sin Complejidad

La Plataforma Moderna de Microservicios

- **Kubernetes Invisible:** Desarrollado sobre AKS; disfrutas de K8s sin lidiar con manifests complejos.
- **Escalado KEDA:** Autoescalado por eventos: mensajes en cola (Service Bus, Kafka) o peticiones HTTP.
- **Escalado a Cero:** Apagado total de réplicas en periodos de inactividad para reducir costes.
- **Descubrimiento DNS:** Resolución de nombres interna entre microservicios del mismo entorno.

Casos de Uso Ideales en Producción

- **APIs Públicas y Privadas:** Enrutamiento Envoy integrado, certificados HTTPS y división de tráfico.
- **Workers de Mensajería:** Procesadores en segundo plano que se activan al llegar mensajes a la cola.
- **Despliegues Blue/Green:** División porcentual de tráfico entre versiones de forma transparente.
- **Alternativa Práctica a K8s:** Para equipos que necesitan elasticidad sin gestionar nodos de clúster.

AMAZON AWS

Contenedores en Amazon Web Services (AWS): Catálogo Central

Amazon ECR (Registry)

Registro de contenedores totalmente gestionado con cifrado KMS, replicación entre regiones de AWS y escaneo con Amazon Inspector.

Amazon ECS (Orquestador)

Orquestador propietario nativo de AWS, maduro e integrado profundamente con IAM, VPC, Application Load Balancers y CloudWatch.

AWS Fargate (Cómputo Serverless)

Motor de ejecución serverless para ECS y EKS. Ejecuta tareas aisladas sin necesidad de aprovisionar ni mantener instancias EC2.

Amazon EKS (Kubernetes)

Servicio gestionado de Kubernetes estándar de la CNCF, predilecto para organizaciones con estrategias multi-cloud o híbridas.

AWS App Runner

Servicio serverless totalmente automatizado para desplegar APIs y aplicaciones web desde repositorio de código o imagen de ECR.

Guía de Decisión en AWS

• ECS + Fargate: La combinación recomendada para el 90% de las empresas.
• EKS: Cuando se requiere compatibilidad CNCF y tooling K8s.

Amazon ECS: EC2 Launch Type vs AWS Fargate Serverless

Modelo EC2 Launch Type (Servidores)

- **Control de Instancias:** Los contenedores corren sobre máquinas EC2 que tú aprovisionas y gestionas.
- **Carga Operativa:** Responsabilidad de parchear el SO anfitrión, actualizar el agente ECS y vigilar discos.
- **Ahorro con Spot:** Permite usar instancias Spot (hasta 90% de descuento para cargas tolerantes a fallos).
- **Hardware Dedicado:** Obligatorio si se requieren GPUs para inferencia de IA o tipos de CPU muy concretos.

Modelo AWS Fargate (Serverless)

- **Cero Instancias EC2:** Defines la tarea con vCPU y RAM; AWS aprovisiona el entorno al vuelo.
- **MicroVMs Firecracker:** Cada tarea corre en su propio sandbox ligero de virtualización en Rust (<5ms de inicio).
- **Mantenimiento Cero:** AWS gestiona los parches de seguridad del host, kernel y runtime OCI.
- **Modelo Predilecto:** La opción recomendada por AWS para reducir drásticamente el coste de mantenimiento.

Amazon ECR: Autenticación IAM y Seguridad en Repositorios

Gobernanza y Seguridad en ECR

- **Autenticación Efímera IAM:** Las contraseñas generadas caducan a las 12 horas (sin secretos estáticos).
- **Resource Policies:** Control granular de acceso entre diferentes cuentas de AWS o roles de servicio.
- **Amazon Inspector:** Detección en tiempo real de vulnerabilidades conocidas (CVEs) en capas y librerías.
- **Tags Inmutables:** Bloqueo declarativo para impedir la sobrescritura accidental de versiones publicadas.

FLUJO DE OPERACIÓN EN AMAZON ECR

```
# 1. Obtener token de login de ECR y pasarlo al CLI de Docker
$ aws ecr get-login-password --region eu-west-1 | \
    docker login --username AWS --password-stdin \
    123456789012.dkr.ecr.eu-west-1.amazonaws.com

# 2. Crear repositorio privado con inmutabilidad y escaneo
$ aws ecr create-repository \
    --repository-name app-backend \
    --image-tag-mutability IMMUTABLE \
    --image-scanning-configuration scanOnPush=true

# 3. Etiquetar y subir la imagen al repositorio privado
$ docker tag mi-api:1.0 \
    123456789012.dkr.ecr.eu-west-1.amazonaws.com/app-backend:1.0
$ docker push \
    123456789012.dkr.ecr.eu-west-1.amazonaws.com/app-backend:1.0
```

GOOGLE CLOUD

Contenedores en Google Cloud Platform (GCP): Cuna de Kubernetes

Pioneros Históricos

Google inventó los cgroups (process containers) en 2006 y desarrolló Borg, el precursor arquitectónico directo de Kubernetes.

Artifact Registry

Almacén universal OCI que unifica imágenes de Docker con paquetes de software (npm, pip, maven) y escaneo de vulnerabilidades.

Google Cloud Run

Plataforma serverless totalmente gestionada para ejecutar contenedores que escalan de 0 a miles de instancias en milisegundos.

Google Kubernetes Engine (GKE)

El estándar de oro en Kubernetes administrado. Modalidad GKE Autopilot para plano de control y plano de datos serverless.

Sandbox gVisor

Núcleo de emulación en Ring 3 desarrollado por Google que intercepta las llamadas al sistema para máxima seguridad multinquilino.

Filosofía Cloud-Native

Google Cloud destaca por su integración nativa con estándares abiertos de la CNCF, evitando el vendor lock-in propietario.

Google Cloud Run: La Excelencia en Serverless Containers



Por Qué Cloud Run Cambió las Reglas

- **Cualquier Lenguaje o Binario:** Ejecuta cualquier imagen Docker que escuche peticiones en el puerto `$PORT`.
- **Alta Concurrencia:** Una única instancia puede atender hasta 1000 peticiones simultáneas en memoria.
- **Escalado Instantáneo:** Reacción en milisegundos ante avalanchas repentinas de tráfico web.
- **Dominio HTTPS y CDN:** Subdominio automático `.run.app` con certificados TLS y red Anycast global.
- **Pago por Petición Exacta:** Facturación al milisegundo; sin tráfico activo el coste es exactamente 0€.

DESPLIEGUE EN CLOUD RUN CON GCLOUD CLI

```
# Desplegar un contenedor en Cloud Run con un único comando
$ gcloud run deploy mi-api-rest \
    --image gcr.io/cloudrun/hello \
    --platform managed \
    --region europe-west1 \
    --allow-unauthenticated \
    --port 8080 \
    --memory 512Mi \
    --min-instances 0 \
    --max-instances 10

# URL lista para producción generada en segundos:
# Service URL: https://mi-api-rest-xyz-ew.a.run.app

# Probar el servicio desplegado
$ curl https://mi-api-rest-xyz-ew.a.run.app
```

Google Artifact Registry: Almacén Universal OCI

Evolución de GCR a Artifact Registry

- **Soporte Multipaquete:** Imágenes OCI, paquetes npm, Python (PyPI), Java Maven y charts Helm unificados.
- **Repositorios Regionales:** Minimiza la latencia de pull ubicando imágenes junto a los clústeres de cómputo.
- **Control de Acceso IAM:** Integración granular con Cuentas de Servicio (Service Accounts) de Google Cloud.
- **Binary Authorization:** Impedir el despliegue en producción si la imagen no cuenta con firma auditada.

FLUJO EN GOOGLE ARTIFACT REGISTRY

```
# 1. Configurar credenciales de Docker para el Registry
$ gcloud auth configure-docker \
    europe-west1-docker.pkg.dev

# 2. Etiquetar la imagen local con el formato regional
$ docker tag mi-app:1.0 \
    europe-west1-docker.pkg.dev/mi-proyecto/repo-apps/mi-app:1.0

# 3. Subir la imagen al repositorio privado
$ docker push \
    europe-west1-docker.pkg.dev/mi-proyecto/repo-apps/mi-app:1.0

# 4. Desplegar directamente en Cloud Run
$ gcloud run deploy mi-app \
    --image europe-west1-docker.pkg.dev/mi-proyecto/repo-apps/mi-app:1.0
```

HISTORIA Y EVOLUCIÓN

Docker Cloud: Análisis Histórico y Evolución hacia el Ecosistema Actual

Qué fue "Docker Cloud" (2015-2018)

- **Origen:** Adquisición de la startup Tutum por parte de Docker Inc. en octubre de 2015.
- **Propósito Original:** Panel web SaaS para desplegar y monitorizar contenedores sobre AWS, Azure y DigitalOcean.
- **Cierre Oficial:** Retirado definitivamente en mayo de 2018 tras el auge arrollador de Kubernetes y CaaS cloud.
- **¿Por qué en el Temario?:** Vestigio histórico de temarios clásicos que debe ser aclarado con rigor por el docente.

Hacia Dónde Evolucionó Docker Inc.

- **Docker Hub Moderno:** El registro público de referencia global con builds automatizados y organizaciones.
- **Docker Desktop:** Plataforma de productividad para desarrolladores (Mac, Windows, Linux) con extensiones.
- **Docker Scout:** Herramienta de análisis de la cadena de suministro, SBOM y remediación de CVEs.
- **Enfoque Estratégico:** Liderar la "experiencia de desarrollo" (DevX) y delegar la infraestructura a la nube.

CRITERIOS DE ARQUITECTURA

¿Cuándo usar CaaS Serverless vs Kubernetes Gestionado?

Elegir CaaS Serverless (Cloud Run, ACI, ACA, Fargate)

- **Equipos Reducidos:** Cero personas dedicadas a administrar infraestructura, redes o parches de clúster.
- **Cargas Web y APIs HTTP:** Microservicios que se benefician directamente del escalado automático a cero.
- **Procesamiento por Lotes:** Tareas que arrancan, procesan datos durante unos minutos y se apagan.
- **Rápido Time-to-Market:** Poner en producción una imagen en 5 minutos con un único comando de CLI.
- **Presupuesto Optimizado:** No pagas por planos de control fijos ni por máquinas virtuales ociosas.

Elegir Kubernetes Gestionado (AKS, EKS, GKE)

- **Topologías Complejas:** Cientos de microservicios interconectados, mallas de servicios (Istio/Linkerd).
- **Control Fino de Red:** Políticas de red L3/L4 personalizadas, volúmenes de bloque y topologías multi-nodo.
- **Ecosistema CNCF Extenso:** Operadores de bases de datos, herramientas GitOps (ArgoCD, Flux) y observabilidad.
- **Estrategia Multi-Cloud:** Evitar vendor lock-in ejecutando los mismos manifests en cualquier nube y on-premise.
- **Equipo DevOps Dedicado:** Capacidad humana para asumir la gestión del clúster y sus ciclos de actualización.

Buenas Prácticas para Contenedores en Producción Cloud

Diseño y Operación Resiliente

- **Arquitectura Stateless:** No guardar estado local; delegar en bases de datos gestionadas y buckets de objetos.
- **Variables de Entorno (12-Factor):** Inyectar configuración y secretos mediante gestores dedicados de la nube.
- **Sondas de Salud (`HEALTHCHECK`):** Declarar verificaciones de salud para que el orquestador reinicie instancias caídas.
- **Logs a stdout / stderr:** Salida directa a consola para su recolección automática en CloudWatch o Cloud Logging.

Seguridad y Control de Costes

- **Prohibido Secretos en Build:** Jamás escribir credenciales o llaves API en el Dockerfile ni durante el build.
- **Usuario No-Root:** Declarar `USER appuser` en la imagen para mitigar riesgos de escape del contenedor.
- **Límites de CPU y Memoria:** Declarar topes en cada servicio para evitar colapsos por consumo descontrolado.
- **Límites de Autoescalado:** Fijar `max-instances` para prevenir sorpresas en la factura mensual ante ataques DoS.

Laboratorio 4: Registries Cloud y Despliegue CaaS

Objetivos del Laboratorio 4

- **Paso 1 · Autenticación:** Conectar el cliente local de Docker al registry mediante credenciales seguras.
- **Paso 2 · Etiquetado Canónico:** Generar la referencia completa `<registry>/<namespace>/<app>:<version>`.
- **Paso 3 · Publicación (Push):** Subir las capas de la imagen y verificar la deduplicación en el servidor.
- **Paso 4 · Auditoría Remota:** Comprobar en el panel web el digest SHA256 y el escaneo de vulnerabilidades.
- **Paso 5 · Despliegue Cloud:** Instanciar el contenedor en un entorno CaaS (ACI / Cloud Run / Registry local).
- **Paso 6 · Verificación Pública:** Probar el endpoint público asignado mediante navegador o `curl`.
- **Paso 7 · Despliegue Continuo:** Publicar la versión 1.0.1 y actualizar el servicio sin pérdida de tráfico.

RESOLUCIÓN DEL LABORATORIO 4 (REGISTRY & PUSH)

```
# 1. Simulación de Registry local v2 (o usar Docker Hub / Cloud)
$ docker run -d -p 5000:5000 --name registry-local registry:2

# 2. Compilar imagen de prueba con tag semántico
$ docker build -t mi-api:1.0.0 .

# 3. Etiquetar para el registro destino
$ docker tag mi-api:1.0.0 localhost:5000/servicios/mi-api:1.0.0

# 4. Enviar capas al registro remoto
$ docker push localhost:5000/servicios/mi-api:1.0.0

# 5. Auditar el catálogo del registro vía API REST OCI
$ curl http://localhost:5000/v2/_catalog
{"repositories":["servicios/mi-api"]}

# 6. Desplegar contenedor descargando desde el registry
$ docker run -d -p 8080:80 localhost:5000/servicios/mi-api:1.0.0
$ curl -I http://localhost:8080
```

04

MÓDULO 4 · CONCLUSIÓN DÍA 3

RESUMEN DE LA JORNADA 3

- CaaS y Serverless: Ejecución elástica de contenedores con escalado a cero.
- Registries OCI: Inmutabilidad de tags, escaneo de CVEs y autenticación IAM.
- Nubes Públicas: Azure Container Apps, AWS Fargate y Google Cloud Run.
- Criterio de Arquitectura: Cuándo utilizar CaaS Serverless vs clúster K8s.
- Laboratorio 4 completado con éxito por los alumnos.

Conceptos consolidados: CaaS, Serverless Containers, Azure (ACR, ACI, ACA), AWS (ECR, ECS, Fargate), Google Cloud (Cloud Run, Artifact Registry) y Registries OCI.
Próxima Jornada: Módulo 5 (Orquestación a escala, Kubernetes y Taller Final Integrador).

05

MÓDULO 5 · 5 HORAS LECTIVAS

ORQUESTACIÓN Y KUBERNETES

Módulo 5: ¿Por qué Orquestar?, Swarm vs K8s vs Nomad, Arquitectura K8s, Primitivas y Taller Integrador.
Jornada 4 de 4 · 5 Horas Lectivas (20h totales)

- Límites de Docker Standalone y la necesidad de orquestación.
- Comparativa de Orquestadores: Docker Swarm vs K8s vs Nomad.
- Arquitectura de Kubernetes: Control Plane y Worker Nodes.
- El Modelo Declarativo y los Bucles de Reconciliación continua.
- Primitivas básicas de K8s: Pods, Deployments y Services.
- Laboratorio 5: Despliegue, auto-recuperación y Rolling Update.

¿Por qué Orquestar? Los Límites Críticos de Docker Standalone

Los Límites de un Solo Host

- **Punto Único de Fallo (SPOF):** Si el servidor cae por hardware o red, todos los servicios colapsan.
- **Límite de Escalado Vertical:** Un único servidor tiene un tope físico de CPU y memoria RAM finita.
- **Sin Auto-Recuperación Multi-Host:** `--restart always` no puede mover el contenedor a otro servidor sano.
- **Gestión Manual Inviable:** Monitorear y actualizar 100 contenedores manualmente en múltiples IPs es imposible.

La Promesa del Orquestador

- **Abstracción de Clúster:** Unifica decenas de servidores en un único grupo lógico de cómputo y red.
- **Auto-Sanación (Self-Healing):** Detecta la muerte de un nodo y reprograma los contenedores en nodos sanos.
- **Balanceo de Carga L4/L7:** Distribución inteligente del tráfico entrante entre todas las réplicas activas.
- **Rolling Updates sin Downtime:** Actualizaciones continuas contenedor a contenedor sin caída de servicio.

PILARES DE ORQUESTACIÓN

Las 5 Capacidades Clave de un Orquestador Moderno

1

1. Scheduling

Evalúa requisitos de CPU, memoria, afinidad y restricciones para elegir el nodo óptimo donde arrancar cada contenedor.

2

2. Auto-Sanación

Monitoriza continuamente sondas de liveness; destruye y recrea automáticamente contenedores bloqueados o caídos.

3

3. Auto-Escalado

Aumenta o reduce el número de réplicas en caliente basándose en métricas de consumo de CPU, RAM o tráfico.

4

4. Service Discovery

Proporciona una IP virtual fija y DNS interno estable, balanceando el tráfico de red entre las réplicas disponibles.

5

5. Rolling Updates

Despliegues progresivos sin parada de servicio con capacidad de rollback inmediato si se detectan anomalías.

COMPARATIVA DE MERCADO

Comparativa de Orquestadores: Docker Swarm vs K8s vs HashiCorp Nomad

Docker Swarm

- **Filosofía:** Simplicidad radical e integración nativa en el motor Docker.
- **Curva de Aprendizaje:** Muy baja (reutiliza la sintaxis conocida de Docker Compose).
- **Despliegue:** Se inicializa en 10 segundos con `docker swarm init`.
- **Ecosistema:** Modesto, desarrollo mantenido principalmente por Mirantis.
- **Caso de Uso Ideal:** Equipos pequeños y clústeres medianos (1 a 15 servidores).

Kubernetes (K8s)

- **Filosofía:** Estándar industrial absoluto de la CNCF respaldado por la industria.
- **Curva de Aprendizaje:** Alta / Compleja (gran variedad de APIs y objetos declarativos).
- **Despliegue:** Complejo on-premise; simple en Cloud gestionado (AKS, EKS, GKE).
- **Ecosistema:** El mayor del software mundial (Helm, ArgoCD, Istio, Prometheus).
- **Caso de Uso Ideal:** Microservicios empresariales y arquitecturas a gran escala.

HashiCorp Nomad

- **Filosofía:** Orquestador flexible, ligero y declarativo en un solo binario Go.
- **Flexibilidad de Cargas:** Corre contenedores Docker, binarios compilados y VMs QEMU.
- **Simplicidad Operativa:** Sin dependencias externas; integra con Consul y Vault.
- **Ecosistema:** Integración nativa con la suite de HashiCorp (Terraform).
- **Caso de Uso Ideal:** Entornos híbridos que combinan contenedores con software legacy.

Docker Swarm a Fondo: El Orquestador Integrado en el Motor

Arquitectura y Routing Mesh de Swarm

- **Nativo en Docker Engine:** Sin agentes externos; se activa con un único comando en el host.
- **Nodos Manager y Worker:** Managers mantienen quórum Raft; Workers ejecutan contenedores.
- **Servicios Replicados:** `docker service` declara réplicas deseadas en lugar de contenedores sueltos.
- **Ingress Routing Mesh:** Los puertos se publican en TODOS los nodos; el tráfico se rutea internamente vía IPVS.
- **mTLS Automático:** Comunicaciones entre nodos cifradas con certificados TLS mutuos autogestionados.

COMANDOS ESENCIALES DE DOCKER SWARM

```
# 1. Inicializar el nodo actual como Manager del clúster
$ docker swarm init
Swarm initialized: current node is now a manager.

# 2. Desplegar servicio web replicado con 3 réplicas
$ docker service create --name web-cluster \
    --replicas 3 \
    -p 80:80 \
    nginx:alpine

# 3. Escalar el número de réplicas en caliente
$ docker service scale web-cluster=6

# 4. Actualización progresiva (Rolling Update)
$ docker service update --image nginx:1.25-alpine web-cluster

# 5. Salir del modo Swarm y restaurar modo standalone
$ docker swarm leave --force
```

Introducción a Kubernetes: El Timonel del Ecosistema Cloud Native

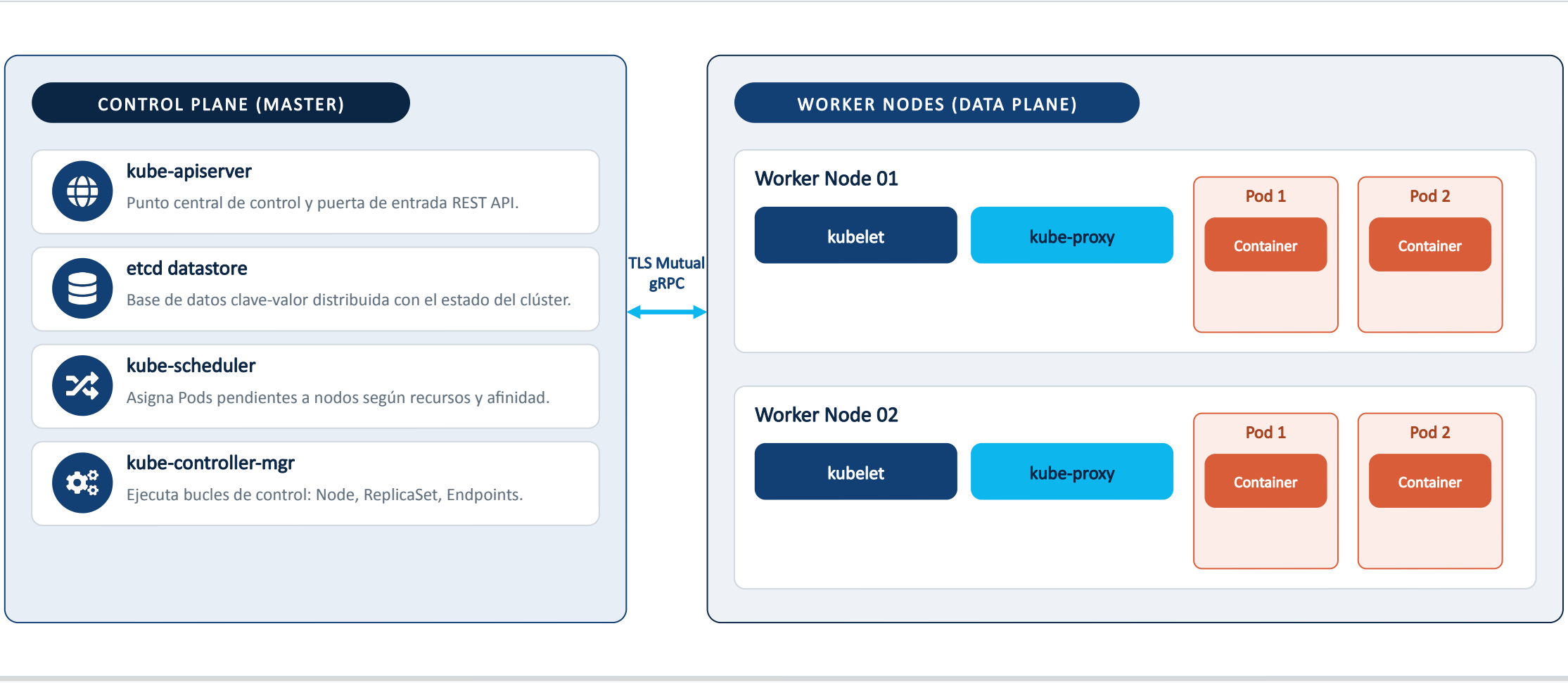
Historia y Misión de Kubernetes

- **Herencia de Google Borg:** Diseñado por ingenieros de Google basándose en 10 años de experiencia interna.
- **Donación a la CNCF:** Gobernanza comunitaria y abierta bajo el paraguas de la Linux Foundation.
- **Sustrato Universal:** El sistema operativo distribuido de la nube soportado por todos los grandes clouds.
- **Estándar OCI:** Ejecuta exactamente las mismas imágenes compiladas con Dockerfile sin modificaciones.

Principios de Diseño de K8s

- **Modelo Declarativo:** Se declara el estado final deseado en ficheros YAML (no comandos imperativos).
- **Inmutabilidad:** Los Pods no se parchean en caliente; se reemplazan por instancias nuevas.
- **API-Driven Architecture:** Todos los componentes interactúan exclusivamente mediante llamadas REST al API Server.
- **Extensibilidad con CRDs:** Capacidad de ampliar la plataforma con controladores y operadores a medida.

Arquitectura de Kubernetes: Control Plane y Worker Nodes



CONTROL PLANE

El Plano de Control (Control Plane) al Detalle

Componentes del Master Node

- **`kube-apiserver`:** Puerta de entrada única al clúster. Expone la API REST, autentica usuarios y valida YAMLs.
- **`etcd datastore`:** Base de datos clave-valor distribuida (Raft). La única fuente de la verdad del estado del clúster.
- **`kube-scheduler`:** Detecta Pods recién creados sin nodo asignado y elige el servidor óptimo según recursos.
- **`kube-controller-manager`:** Ejecuta bucles de control (Node, Deployment, Endpoints) que fuerzan el estado deseado.

INTERACCIÓN CON EL API SERVER VÍA KUBECTL

```
# Toda interacción con K8s se realiza vía el CLI kubectl
# que emite llamadas REST seguras hacia kube-apiserver (:6443)

# Listar nodos del clúster y sus roles
$ kubectl get nodes
NAME             STATUS   ROLES           AGE   VERSION
k8s-control-01   Ready    control-plane   30d   v1.29.2
k8s-worker-01    Ready    <none>          30d   v1.29.2
k8s-worker-02    Ready    <none>          30d   v1.29.2

# Obtener información del plano de control
$ kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:6443
CoreDNS is running at https://127.0.0.1:6443/api/v1/namespaces/...
```

Los Nodos de Trabajo (Worker Nodes) al Detalle

Componentes en cada Servidor Worker

- **`kubelet`:** Agente supervisor del nodo. Recibe PodSpecs del API server y vigila que los contenedores corran.
- **`kube-proxy`:** Gestor de red local. Programa reglas iptables / IPVS para balancear peticiones hacia los Pods.
- **Container Runtime (CRI):** Motor OCI de bajo nivel (`containerd` o `CRI-O`) que descarga imágenes y ejecuta procesos.
- **Pods de Aplicación:** La carga de trabajo real (contenedores de Node, Python, bases de datos) corriendo en el host.

FLUJO INTERNO EN EL WORKER NODE

```
# 1. kube-scheduler asigna un Pod a "k8s-worker-01"
# 2. kubelet en worker-01 detecta la nueva asignación
# 3. kubelet invoca a containerd vía gRPC (CRI):
#    - PullImage(mi-api:1.0)
#    - CreateContainer()
#    - StartContainer()
# 4. kube-proxy actualiza reglas iptables en el nodo
# 5. kubelet ejecuta periódicamente las sondas de salud:
#    - livenessProbe (¿Sigue vivo el proceso?)
#    - readinessProbe (¿Puede recibir tráfico web?)
```

El Bucle de Reconciliación: La Magia Declarativa de Kubernetes

Filosofía del Bucle de Control Infinito

- **Estado Deseado (Desired):** Declaración explícita en archivos YAML almacenada en etcd (ej. 3 réplicas).
- **Estado Actual (Current):** La realidad de los procesos en ejecución en los Worker Nodes del clúster.
- **Bucle de Reconciliación:** Los controladores ejecutan continuamente el ciclo: `OBSERVAR -> COMPARAR -> ACTUAR`.
- **Auto-Recuperación Automática:** Si un Pod muere o un nodo falla, K8s crea nuevas instancias hasta igualar el estado.

LÓGICA DEL RECONCILIADOR EN KUBERNETES

```
# Pseudocódigo conceptual del Controller Manager:
while True:
    # 1. Leer estado deseado vs estado actual
    desired = read_from_etcd()       # spec.replicas = 3
    current = inspect_active_pods()  # actual_pods = 2

    # 2. Reconciliar si existe discrepancia
    if current != desired:
        diff = calculate_diff(desired, current)
        execute_reconcile(diff)      # Crea 1 Pod nuevo

    wait_for_watch_event()           # Arquitectura reactiva

# ¡Bucle activo 24/7 sin intervención humana!
```

Primitiva Fundamental 1: El Pod (Unidad Atómica de Cómputo)

La Unidad Mínima de Cómputo

- **Sin Contenedores Sueltos:** Kubernetes gestiona, programa y escala Pods, nunca contenedores individuales.
- **Grupo de Contenedores:** Aloja un contenedor principal y opcionalmente contenedores auxiliares (Sidecars).
- **Red Compartida:** Misma dirección IP para todos los contenedores del Pod; comunicación por `localhost`.
- **Almacenamiento Compartido:** Los volúmenes definidos en el Pod son accesibles por todos sus contenedores.

MANIFIESTO YAML DE UN POD (POD.YAML)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mi-pod-web
  labels:
    app: frontend
spec:
  containers:
  - name: nginx-server
    image: nginx:alpine
    ports:
    - containerPort: 80
    resources:
      limits:
        memory: "128Mi"
        cpu: "500m"
```

Primitiva Fundamental 2: Deployments, ReplicaSets y Rollouts



Gestión de Ciclo de Vida y Réplicas

- **Prohibido Pods Sueltos:** En producción los Pods siempre se encapsulan dentro de un **Deployment**.
- **Controlador ReplicaSet:** Garantiza que siempre exista el número exacto de réplicas sanas (`spec.replicas`).
- **Estrategia RollingUpdate:** Actualización progresiva pod a pod sin un solo segundo de corte de servicio.
- **Rollback Inmediato:** Vuelta atrás instantánea con `kubectl rollout undo deployment/nombre`.

MANIFIESTO YAML DE UN DEPLOYMENT

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-pedidos
spec:
  replicas: 3
  selector:
    matchLabels:
      app: pedidos
  template:
    metadata:
      labels:
        app: pedidos
    spec:
      containers:
      - name: api
        image: mi-repo/pedidos:2.0
        ports:
        - containerPort: 3000
```

Primitiva Fundamental 3: Services y Balanceo de Carga L4

Abstracción de Red Estable

- **El Problema de las IPs Efímeras:** Los Pods nacen y mueren continuamente con IPs cambiantes.
- **ClusterIP Fija y DNS:** El Service ofrece una IP virtual permanente y resolución por nombre interno.
- **Selectores por Etiquetas:** Vinculación dinámica mediante labels (`app: pedidos`).
- **Tipos Principales:** `ClusterIP` (interno), `NodePort` (puerto en nodos), `LoadBalancer` (Cloud).

MANIFIESTO YAML DE UN SERVICE

```yaml
apiVersion: v1
kind: Service
metadata:
  name: servicio-pedidos
spec:
  type: ClusterIP
  selector:
    # Conecta con los Pods que tengan este label
    app: pedidos
  ports:
  - port: 80         # Puerto de escucha del Service
    targetPort: 3000 # Puerto real del contenedor

# Consumible desde cualquier Pod mediante DNS interno:
# http://servicio-pedidos:80
```

Configuración y Persistencia: ConfigMaps, Secrets y PVCs

ConfigMaps y Secrets (Configuración Externa)

- **ConfigMaps:** Variables de entorno y ficheros de configuración desacoplados de la imagen.
- **Secrets:** Almacenamiento de credenciales y certificados con cifrado en reposo KMS.
- **Inyección Flexible:** Inyectables como variables de entorno (`env`) o montados como carpetas.
- **Portabilidad:** La misma imagen corre en Dev, Staging y Prod cambiando solo el ConfigMap.

Persistencia: PVCs, PVs y StorageClasses

- **PersistentVolumeClaim (PVC):** Petición formal de almacenamiento declarada por el Pod (ej. 50GB).
- **PersistentVolume (PV):** El volumen físico aprovisionado en disco de nube (EBS, Azure Disk).
- **StorageClass Dinámica:** Crea el disco cloud al vuelo al detectar la creación de un nuevo PVC.
- **StatefulSets:** Controlador especializado para bases de datos que requieren identidad fija.

De Docker a Kubernetes: Diccionario y Tabla de Equivalencias

Concepto en Docker Standalone / Compose	Homólogo en Kubernetes (K8s)	Función y Propósito Técnico
docker run -d ...	Pod / Deployment	Instancia de proceso en ejecución (Pod) y controlador de réplicas.
docker build -t mi-app .	Imagen OCI (Idéntica)	El proceso de build es 100% el mismo; K8s descarga la misma imagen.
-p 8080:80 (Mapeo de puertos)	Service (ClusterIP / LoadBalancer)	Descubrimiento DNS y balanceo de tráfico hacia los Pods.
docker network create ...	CNI Plugin + NetworkPolicy	Red plana de clúster donde todos los Pods tienen IP única.
-v mi_vol:/var/lib/data	PersistentVolumeClaim (PVC)	Almacenamiento permanente gestionado por StorageClasses.
docker compose up	kubectl apply -f / Helm Chart	Despliegue declarativo de la topología completa de servicios.

Entornos Locales para Practicar y Dominar Kubernetes

Opciones para Practicar en tu Portátil

- **Docker Desktop K8s:** Activar en Settings > Kubernetes > Enable. Un clúster de 1 nodo listo en 1 minuto.
- **Kind (K8s in Docker):** Corre nodos de K8s dentro de contenedores Docker en tu host. Excelente para CI/CD.
- **Minikube:** Herramienta histórica oficial de la comunidad K8s con addons preconfigurados.
- **K3s (Rancher/SUSE):** Distribución ultraligera (<100MB) en un solo binario Go para Edge y desarrollo.

CLÚSTER CON KIND (KUBERNETES IN DOCKER)

```
# 1. Crear un clúster local de prueba en 30 segundos
$ kind create cluster --name curso-docker
Creating cluster "curso-docker" ...
 ✓ Ensuring node image (kindest/node:v1.29.2)
 ✓ Preparing nodes
 ✓ Starting control-plane
Set kubectl context to "kind-curso-docker"

# 2. Verificar que el clúster responde
$ kubectl get nodes
NAME                        STATUS   ROLES           AGE
curso-docker-control-plane   Ready    control-plane   45s

# 3. Destruir el clúster al finalizar la práctica
$ kind delete cluster --name curso-docker
```

LABORATORIO PRÁCTICO · TALLER INTEGRADOR FINAL

Laboratorio 5: Despliegue, Auto-Recuperación y Rolling Update en K8s

Objetivos del Taller Integrador

- **Paso 1 · Verificación:** Comprobar acceso y salud de nodos con `kubectl get nodes`.
- **Paso 2 · Manifiesto Deployment:** Declarar 3 réplicas con límites de memoria (`128Mi`) y CPU (`500m`).
- **Paso 3 · Despliegue Declarativo:** Aplicar con `kubectl apply` y observar la creación simultánea de Pods.
- **Paso 4 · Exposición con Service:** Publicar el servicio mediante un Service con balanceo de carga.
- **Paso 5 · Prueba de Auto-Sanación:** Eliminar un Pod a mano y constatar que el reconciliador crea otro al instante.
- **Paso 6 · Rolling Update:** Actualizar la versión de la imagen sin un solo segundo de corte de tráfico.

RESOLUCIÓN DEL LABORATORIO 5 EN KUBERNETES

```
# 1. Aplicar Deployment y Service declarativamente
$ kubectl apply -f deployment.yaml
$ kubectl apply -f service.yaml

# 2. Inspeccionar los Pods creados por el ReplicaSet
$ kubectl get pods -l app=pedidos
NAME                            READY   STATUS    RESTARTS
api-pedidos-7c489f6d7b-4k2x8   1/1     Running   0
api-pedidos-7c489f6d7b-9m5z1   1/1     Running   0
api-pedidos-7c489f6d7b-r2w9p   1/1     Running   0

# 3. Demostración de Auto-Sanación (Self-Healing)
$ kubectl delete pod api-pedidos-7c489f6d7b-4k2x8
# ¡El ReplicaSet crea un sustituto inmediatamente!

# 4. Actualización progresiva en caliente (Rolling Update)
$ kubectl set image deployment/api-pedidos api=nginx:1.25-alpine
$ kubectl rollout status deployment/api-pedidos
deployment "api-pedidos" successfully rolled out
```

¡ENHORABUENA! CURSO DOCKER FUNDAMENTALS COMPLETADO



Día 1 · Motor y OCI

Aislamiento de kernel mediante Namespaces y Cgroups, arquitectura desacoplada containerd/runc, creación de imágenes OCI optimizadas y Multi-Stage builds.



Día 2 · Redes y Almacenamiento

Switch virtual docker0, pares veth, resolución DNS embebida 127.0.0.11, reglas iptables NAT, Overlay2 con Copy-on-Write y persistencia con Named Volumes.



Día 3 · Despliegue en la Nube

Evolución a CaaS y Serverless Containers (Azure ACI/ACA, AWS Fargate, Google Cloud Run) y gobernanza de Container Registries privados con firmas e inmutabilidad.



Día 4 · Orquestación y K8s

Gestión distribuida, Swarm vs Nomad vs K8s, arquitectura Control Plane / Workers, bucles de reconciliación continua, Pods, Deployments y Services.